

# HW1

Team nineteen

<https://cal-cs184-student.github.io/hw1-nineteen-writeup/>

The renderer turns SVG shapes into pixels and adds transforms, color blending, and textures. More samples helped with jagged edges, while bilinear filtering and mipmaps helped with blocky or noisy textures.

## Task 1

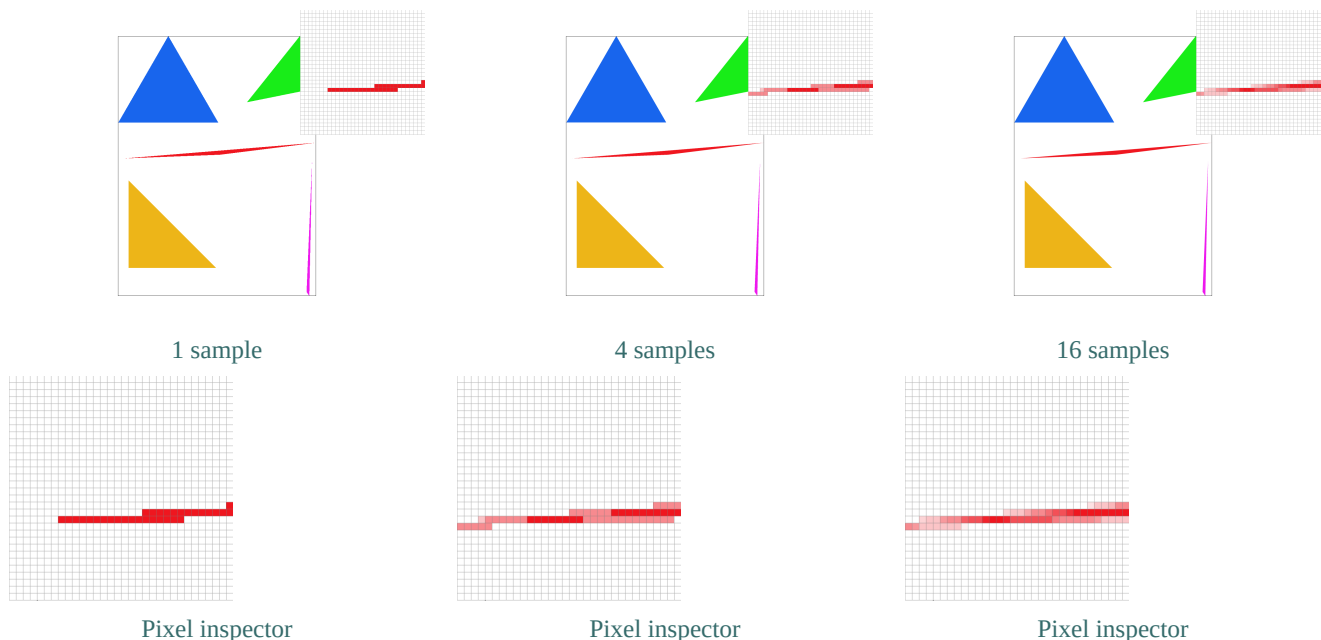
The loop starts with a box around the triangle, clipped to the screen, so it doesn't have to check every pixel on the screen. Inside that box, it checks the center of each pixel, half a pixel across and down from its corner. Three edge tests tell whether the point is inside. If the results are all nonnegative or all nonpositive, the point gets drawn. Accepting either sign makes both vertex orders work, and accepting zero includes points on the edges. Triangles with zero area are skipped.

There are only three edge checks per sample, so the work grows with the number of samples in the bounding box. The picture labeled 1 sample below is also the result for this part.

## Task 2

With supersampling, each pixel gets several sample points instead of one. Four samples use a 2 by 2 grid, and sixteen use a 4 by 4 grid. Each point sits in the middle of its small square. The sample buffer stores one color per sample, grouped by pixel. So sixteen samples means storing sixteen colors for every pixel. The buffer is resized when the window size or sample rate changes, and starts white each frame.

A triangle only fills the samples it covers. Points and lines fill all samples of their pixel. Once everything is drawn, the samples are averaged to get one final color per pixel.

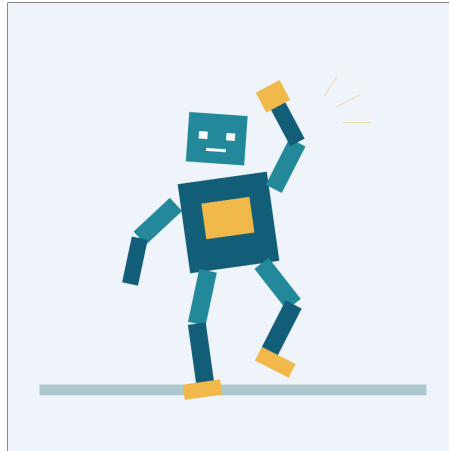


These are basic/test4.svg at the default view, with the inspector in the same spot. The red tip looks rough at 1 sample. At 4 and 16 samples, the edge has lighter red pixels and looks smoother. Those pixels are only partly covered, so averaging mixes the red with the white background.

### Task 3

Translation moves the shape, scaling changes its size, and rotation turns it. The code uses 3 by 3 matrices for these. For rotation, the angle is converted from degrees to radians before using sine and cosine.

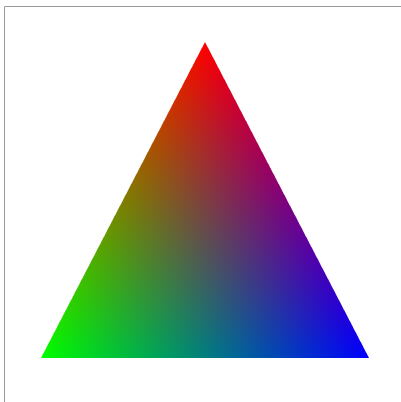
The idea for the robot was to have it wave while taking a step. Its body leans a little, with one arm up and one knee bent. The arm and leg parts are in nested groups, so moving a joint also moves the parts attached to it. For example, the forearm follows the shoulder because it is inside the upper-arm group. The file is `my_robot.svg`.



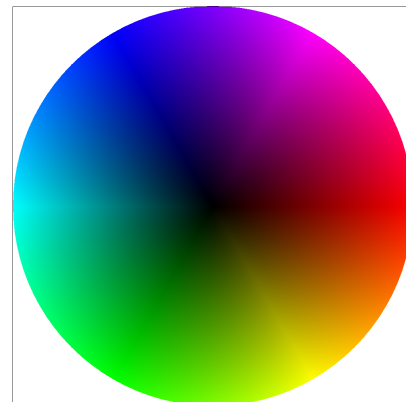
`my_robot.svg`, 16 samples per pixel.

### Task 4

Barycentric coordinates give a point three weights, one for each vertex. They add up to 1. Each weight comes from the area of the opposite small triangle divided by the full triangle area. Each vertex color is multiplied by its weight, then the three results are added together. At a vertex, only that vertex's color is used. Equal weights give an equal mix of the three colors.



RGB triangle, 1 sample



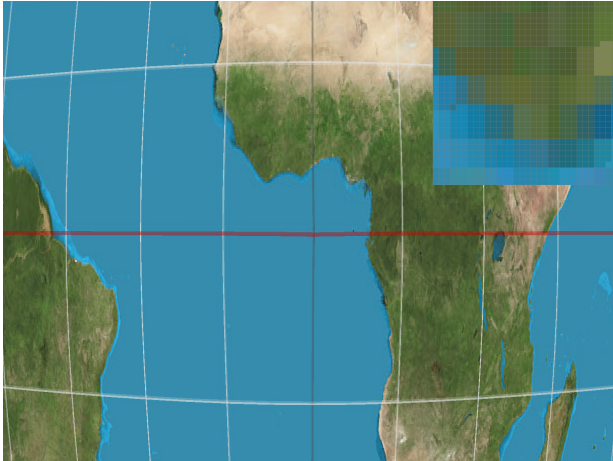
`basic/test7.svg`, default view, 1 sample

In the left picture, moving toward the top makes the color more red because the red weight gets larger. The bottom edge only mixes green and blue. The color wheel on the right uses the same blending across its triangles.

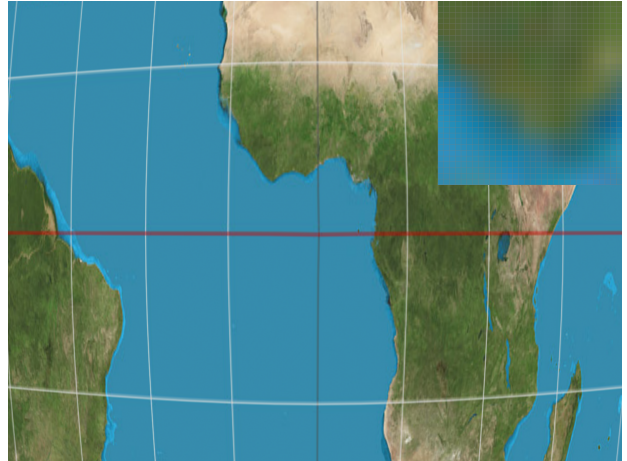
## Task 5

UV coordinates tell where a sample lands in the texture image. The barycentric weights blend the vertex UVs to find that position, which is then scaled to the texture's pixel coordinates. Indices at the edges are clamped so they don't go outside the image.

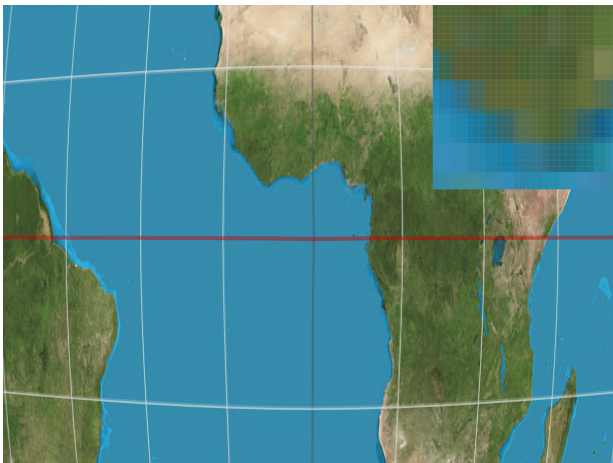
Nearest sampling just rounds to one texel and uses its color. Bilinear takes the four nearby texels and blends them based on how close the sample is to each one. It blends across each row first, then between the two rows.



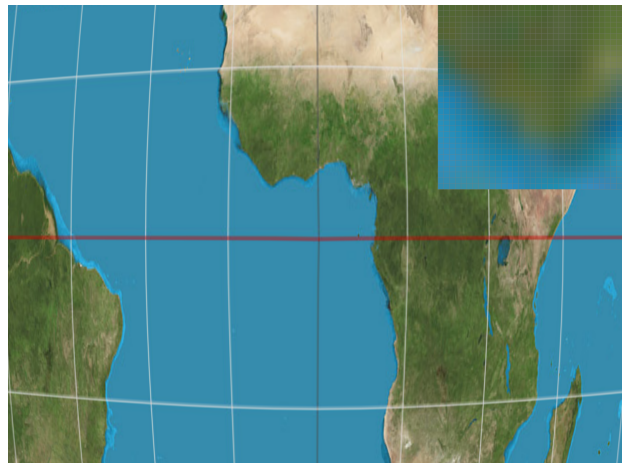
Nearest, 1 sample



Bilinear, 1 sample



Nearest, 16 samples



Bilinear, 16 samples

These show texmap/test1.svg with L\_ZERO and the same zoom and inspector spot. Nearest at 1 sample looks blocky around the coast. Going to 16 samples softens those blocks. Bilinear already looks smoother at 1 sample, so going to 16 changes it less.

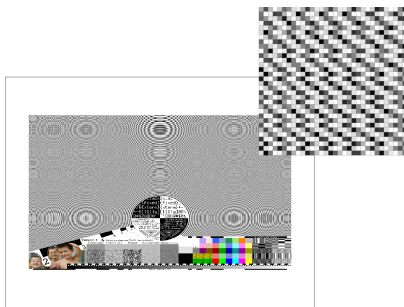
Zooming in makes the difference easier to see. One texel covers several screen pixels, so nearest repeats the same color over a block while bilinear blends into the next color. This matters most around sharp color changes. When zoomed far out, bilinear can still leave aliasing.

## Task 6

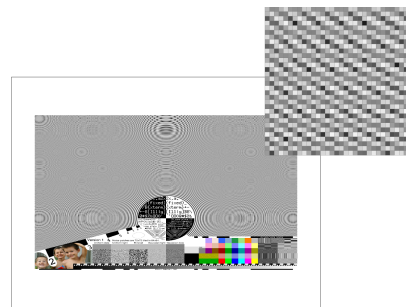
Mipmaps are smaller, averaged texture images. The code compares a sample's UV with the UV one screen pixel right and down. It multiplies u changes by the original width and v changes by the height. The longer change estimates texels per pixel. Each doubling raises the level by one: two texels gives level 1, four gives level 2. The result stays within the available levels.

L\_ZERO stays at level 0. L\_NEAREST rounds to the closest level. L\_LINEAR blends the two levels around the result. Nearest or bilinear pixel sampling works with any of these. Bilinear pixels with linear levels is trilinear filtering.

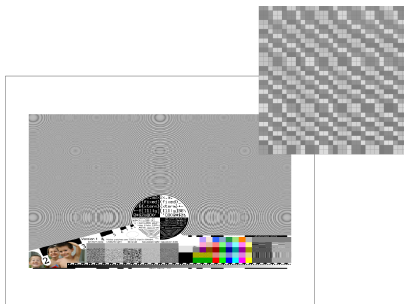
More screen samples help with edges, but take more time and buffer memory. Bilinear reads four texels instead of one, without needing another image, and helps when zoomed in. Mipmaps help when zoomed out, but a square power-of-two texture needs about one third more memory for its mip chain. Blending two levels takes more work than picking one, but avoids sudden level changes.



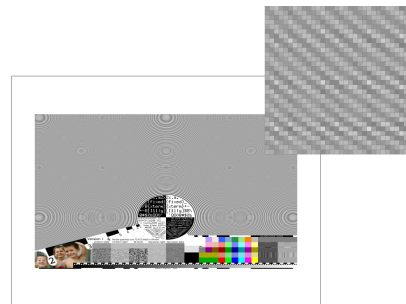
L\_ZERO + P\_NEAREST



L\_ZERO + P\_LINEAR



L\_NEAREST + P\_NEAREST



L\_NEAREST + P\_LINEAR

These use Zone plate boys at 1 sample and the same view. Level 0 has false patterns, and bilinear only reduces them a little. Using the nearest mip level removes more fine detail. Adding bilinear looks smoothest here. The texture covers about 1.8 texels per screen pixel, so the nearest mip level is 1.

The last mip levels initially came out black because the loop skipped them. Making the loop fill every level fixed this.

Some reference pixels still differ on thin edges and the color wheel. The starter also does not handle the partly transparent strokes in the lion and sphere.

Image: AlbertCahalan, Zone plate boys, CC BY-SA 3.0. The original PNG and filtered screenshots use this license. Codex helped with code, tests, and writing.

## Task 1

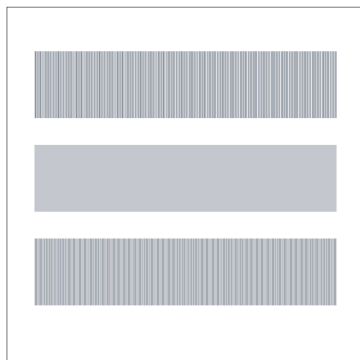
Extra credit: the faster version finds the part of each pixel row that the triangle can touch. It only checks samples in that smaller range. A one-pixel margin keeps edge samples safe, and very large coordinates use the original box. The edge test and colors stay the same. B switches between the two versions.

| Scene      | Samples | Box (ms) | Rows (ms) |
|------------|---------|----------|-----------|
| test4.svg  | 1       | 0.539    | 0.268     |
| test4.svg  | 16      | 6.000    | 2.975     |
| skinny.svg | 1       | 9.517    | 0.155     |
| skinny.svg | 16      | 99.217   | 1.060     |

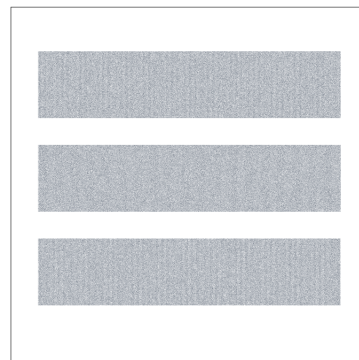
These are CPU times around `svg.draw()`, measured with `clock()` on this Mac at 800 by 800, compiled with `-O3`. Each number is the median of seven batches of twelve draws after a warm-up. Loading, mip generation, clearing, and resolving are outside the timer. Test4 is about twice as fast. The thin triangles improve much more because most of their boxes are empty.

## Task 2

Extra credit: J switches to jittered samples. Each small grid square gets one randomly placed point. The pixel and sample numbers give it a fixed seed, so it stays still across redraws. All triangles use the same positions, which avoids gaps between them. One sample per pixel still uses the center.



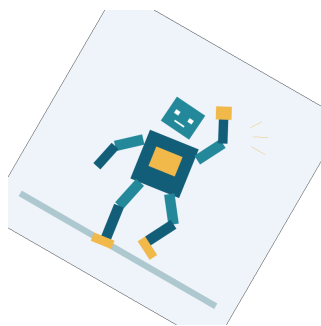
Grid, 16 samples



Jittered, 16 samples

The scene has very thin vertical bars with three slightly different spacings. The grid makes false vertical bands. Jittering breaks those bands into fine grain. It still has noise, but it does not lock onto the repeating bars.

## Task 3

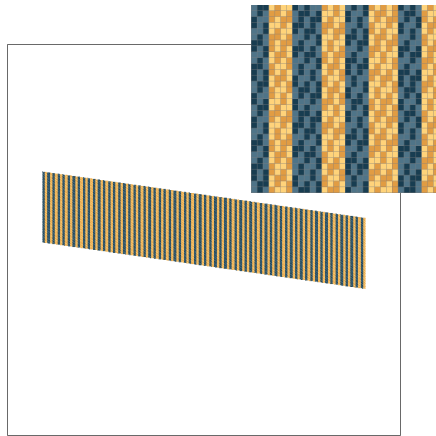


View rotated 30 degrees.

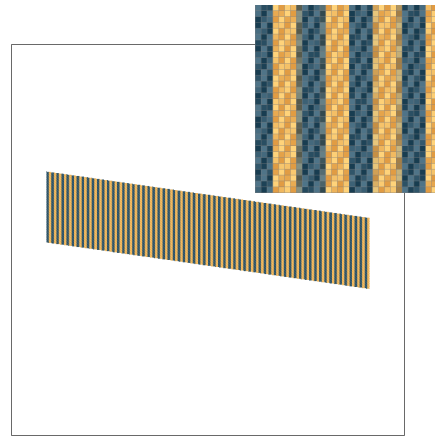
Extra credit: `[` and `]` turn the view by 15 degrees. The SVG first moves into NDC. Then the center moves to the origin, the view rotates, and the center moves back before the screen transform. The canvas border uses the same matrix. Space resets the angle. Dragging is rotated back into SVG space, so it follows the mouse.

## Task 6

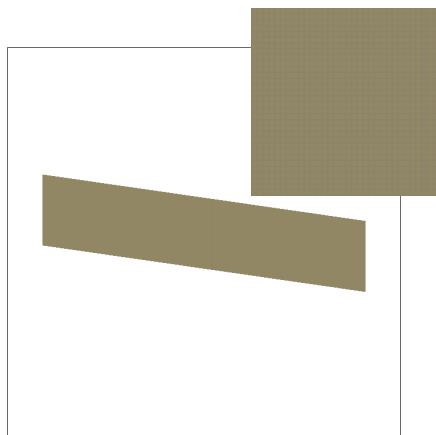
Extra credit: pressing L once after trilinear selects anisotropic filtering. The UV changes give the long and short directions of a pixel in the texture. The short direction chooses the mip level. Several samples are spread along the long direction and averaged. The code uses at most 16 taps, widening the short filter if more would be needed. It reuses the mipmaps and supports both pixel filters.



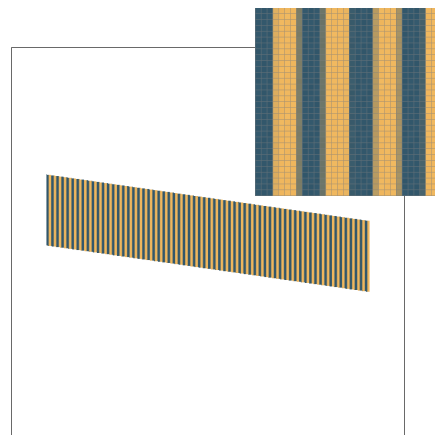
Nearest, level 0



Bilinear, level 0



Trilinear



Anisotropic

All four use one screen sample and the same view. This generated weave is squeezed much more in one direction. Nearest and bilinear leave checker noise. Trilinear removes it but also loses the colored stripes. Anisotropic removes most of the noise while keeping the stripes.

| Method      | CPU time (ms) |
|-------------|---------------|
| Nearest     | 1.074         |
| Bilinear    | 2.389         |
| Trilinear   | 4.955         |
| Anisotropic | 28.938        |

These use the same timing setup as Task 1. Anisotropic is slower because it takes several texture samples per pixel. It needs no extra texture images beyond the mip chain. This is a simple line of taps, so it is still an approximation. The mip choice follows PBRT, linked below.

## Task 7

For the art scene, the robot is camping on another planet. The sky and planet use color triangles. Small polygons make the stars, mountains, tent, and fire. The ring is made from short pieces, drawn behind and in front of the planet.



`competition.svg`, 800 by 800, saved with `S` at 16 samples.

The script `src/generate_competition.py` writes the SVG. Sine and cosine place the planet and ring vertices, and a fixed random seed places the stars. The robot uses nested groups for its waving arm. Everything is made from supported shapes, with no paths.

Codex helped with the extra features, checks, scene generator, and writing.

Texture source | CC BY-SA 3.0 license | PBRT filtering reference